Juan Manuel Castaño Buitrago

Samuel Mejia

Christian David Lopera

Presentado a: Carlos Andres Florez



Noviembre de 2026

Programación III- 01 N

Combinación realizada: ejercicios 1 y 3

Ingeniería en sistemas y computación

1.Enlace del repositorio github: https://github.com/jua12buitrago/Parcial_2.git

2. Ejercicio número 1:

Datos de entrada que se utilizaron:

```
# ciudad_a,ciudad_b,distancia_km
Armenia,Pereira,45
Pereira,Manizales,52
Armenia,Cartago,60
Cartago,Pereira,25
Pereira,Armenia,45
Manizales,Manizales,0
Cartago,Pereira,30
Armenia, Pereira, 45
Armenia, Manizales, 100
Cartago, Manizales, cuarenta
Pereira, Cartago, -15
Armenia,Calarca,6
Calarca,Tebaida,24
Armenia,Tebaida,20
Tebaida,Montenegro,20
Montenegro,Quimbaya,11
Armenia,Tebaida,17
Cali,Palmira,30
Palmira,Cali,30, peaje
```

Salidas obtenidas:

```
Línea 11: Cartago, Manizales, cuarenta -> Distancia no numérica ('cuarenta')
Línea 12: Pereira, Cartago, -15 -> La distancia no puede ser negativa ('-15')
Línea 20: Palmira,Cali,30, peaje -> Formato incorrecto (se esperaban 3 campos)
```

Pruebas de escritorio:

Métrica / Parámetro	Valor Obtenido	Observación / Detalle Técnico
Archivo de Entrada CSV	conexiones.csv	20 líneas totales procesadas perezosamente
Líneas Sintácticamente Válidas	16	Líneas parseadas correctamente por Conexion.parse/1
Errores de Formato / Sintaxis	3	Registrados en errores_conexiones.txt
Conexiones Descartadas (Depuración)	5	Registradas en depuracion_conexiones.txt (Recursión de Cola)
Conexiones Válidas en el Grafo	11	Integradas al mapa no dirigido en Elixir
Pasos para Alcance Completo	4	Pasos 0 a 3 (Se estabiliza en el paso 3 con 8 ciudades)
Rutas Encontradas (Armenia -> Manizales)	3	Ruta 1 (100 km), Ruta 2 (97 km), Ruta 3 (137 km)
Ruta Más Corta (N° Ciudades)	Armenia -> Manizales	2 ciudades
Ruta de Menor Distancia (km)	Armenia -> Pereira -> Manizales	97 km total

Árbol de llamadas recursivas:

```
explorar(Armenia, [Armenia], 0)
  + Manizales (100 km)
  |   explorar(Manizales, [Armenia, Manizales], 100)
  |     -> Destino alcanzado (Ruta 1: Armenia -> Manizales, 2 ciudades, 100 km)
  + Pereira (45 km)
  |   explorar(Pereira, [Armenia, Pereira], 45)
  |     - Armenia: ya visitada, omitir
  |     + Manizales (52 km)
  |     |   explorar(Manizales, [Armenia, Pereira, Manizales], 97)
  |     |     -> Destino alcanzado (Ruta 2: Armenia -> Pereira -> Manizales, 3 ciudades, 97 km)
  |     + Cartago (25 km)
  |         explorar(Cartago, [Armenia, Pereira, Cartago], 70)
  |           - Armenia / Pereira: ya visitadas, omitir (retorna sin ruta)
  + Cartago (60 km)
  |   explorar(Cartago, [Armenia, Cartago], 60)
  |     - Armenia: ya visitada, omitir
  |     + Pereira (25 km)
  |         explorar(Pereira, [Armenia, Cartago, Pereira], 85)
  |           - Armenia / Cartago: ya visitadas, omitir
  |           + Manizales (52 km)
  |               explorar(Manizales, [Armenia, Cartago, Pereira, Manizales], 137)
  |                 -> Destino alcanzado (Ruta 3: Armenia -> Cartago -> Pereira -> Manizales, 4 ciudades, 137 km)
  + Calarca (6 km)
  |   explorar(Calarca, [Armenia, Calarca], 6) -> Conecta a Tebaida (sin llegada a Manizales)
  + Tebaida (20 km)
      explorar(Tebaida, [Armenia, Tebaida], 20) -> Conecta a Montenegro/Calarca (sin llegada a Manizales)
```

Resultados obtenidos:

```
Resultados Obtenidos:
- Número de rutas: 3
- Ruta 1: Armenia -> Manizales (2 ciudades, 100 km)
- Ruta 2: Armenia -> Cartago -> Pereira -> Manizales (4 ciudades, 137 km)
- Ruta 3: Armenia -> Pereira -> Manizales (3 ciudades, 97 km)
- Ruta más corta (menos ciudades): Armenia -> Manizales (2 ciudades)
- Ruta de menor distancia (km): Armenia -> Pereira -> Manizales (97 km)
```

2. Ejercicio número 3:

Datos de entrada que se usaron:

Entrada de la sopa:

ELIXIRR
PCUNMOI
ARODELX
SAÑSBUI
OGOATNL
TEMFVAE

Entrada de las palabras a buscar:

palabras a buscar
ELIXIR
costa
Elixir
PYTHON

Salidas obtenidas:

```
PS C:\Users\User\OneDrive\Documentos\1.Repos\Parcial2_Programacion3> elixir sopaMain.exs
Iniciando ejecución...
Cantidad de letras leídas en la sopa: 42
Palabras a buscar: ["ELIXIR", "COSTA", "PYTHON"]
Dimensiones de la matriz {filas, columnas}: {6, 7}
Datos válidos, generando reporte...
¡Búsqueda finalizada! Resultados escritos en resultado_sopa.txt
PS C:\Users\User\OneDrive\Documentos\1.Repos\Parcial2_Programacion3>
```

Pruebas de escritorio:

#	Función	Valores de entrada	Decisión	Resultado
VALIDACIÓN: palabra_posible? (letras de la sopa: A=4, C=1, O=3, S=2, T=2, Ñ=1, H=0, Y=0, ...)				
1	comparar_frecuencias	PYTHON, letra H	necesita 1, la sopa tiene 0: 1 ≤ 0 es falso	**false: descartada (faltan H, Y)**
2	comparar_frecuencias	ÑAÑA, letra A	necesita 2, hay 4: 2 ≤ 4 es verdadero	sigue con la letra Ñ
3	comparar_frecuencias	ÑAÑA, letra Ñ	necesita 2, hay 1: 2 ≤ 1 es falso	**false: descartada (necesita 2, hay 1)**
4	comparar_frecuencias	COSTA, letras A, C, O, S, T	cada una necesita 1; hay 4, 1, 3, 2, 2: todas alcanzan	**lista vacía: true, pasa a la búsqueda**
BÚSQUEDA: buscar_todas(sopa, 6, 7, "COSTA")				
5	recorrer_filas y recorrer_columnas	Filas 0 a 5, columnas 0 a 6 (42 celdas)	Solo {1,1} tiene C; las otras 41 fallan en la 1.ª letra	Solo {1,1} pasa a probar direcciones
6	probar_direcciones	Desde {1,1}: las 8 direcciones	7 fallan en la 2.ª letra: derecha U, izquierda P, abajo R, arriba L, abajo-izq. A, arriba-der. I, arriba-izq. E (ninguna es O)	Solo abajo-derecha {1,1} sigue
7	comparar_letras	{1,1}, pendientes [C,O,S,T,A], ruta []	C = C coincide	avanza a {2,2}, ruta [{1,1}]
8	comparar_letras	{2,2}, pendientes [O,S,T,A]	O = O coincide	avanza a {3,3}, ruta [{2,2}, {1,1}]
9	comparar_letras	{3,3}, pendientes [S,T,A]	S = S coincide	avanza a {4,4}, ruta [{3,3},...]
10	comparar_letras	{4,4}, pendientes [T,A]	T = T coincide	avanza a {5,5}, ruta [{4,4},...]
11	comparar_letras	{5,5}, pendientes [A]	A = A coincide	avanza a {6,6}, ruta [{5,5},...]
12	comparar_letras	{6,6}, pendientes []	caso base: no quedan letras	retorna la ruta (en orden inverso)
13	revertir_lista	[{5,5},{4,4},{3,3},{2,2},{1,1}]	invierte la lista	**[{1,1},{2,2},{3,3},{4,4},{5,5}]**
RESULTADO FINAL				
14	procesar_palabras	COSTA: 1 ruta	rutas ≠ [] y encontradas pasa de 1 a 2	**Encontrada: Sí (1 forma), abajo-derecha de {1,1} a {5,5}**

Arbol de llamadas recursivas:

```
buscar_todas(COSTA)
+ recorrer_filas(fila 0)
|  - recorrer_columnas({0,0}) = E != C: 8 direcciones fallan
|  - recorrer_columnas({0,1}) = L != C: 8 direcciones fallan
|  - recorrer_columnas({0,2}) = I != C: 8 direcciones fallan
|  - recorrer_columnas({0,3}) = X != C: 8 direcciones fallan
|  - recorrer_columnas({0,4}) = I != C: 8 direcciones fallan
|  - recorrer_columnas({0,5}) = R != C: 8 direcciones fallan
|  - recorrer_columnas({0,6}) = R != C: 8 direcciones fallan
+ recorrer_filas(fila 1)
|  - recorrer_columnas({1,0}) = P != C: 8 direcciones fallan
|  + recorrer_columnas({1,1}) = C OK
|  |  + direccion {0,1}
|  |  |  {1,1} = C OK
|  |  |  {1,2} = "U" != O -> nil
|  |  + direccion {0,-1}
|  |  |  {1,1} = C OK
|  |  |  {1,0} = "P" != O -> nil
|  |  + direccion {1,0}
|  |  |  {1,1} = C OK
|  |  |  {2,1} = "R" != O -> nil
|  |  + direccion {-1,0}
|  |  |  {1,1} = C OK
|  |  |  {0,1} = "L" != O -> nil
|  |  + direccion {1,1}
|  |  |  {1,1} = C OK
|  |  |  {2,2} = O OK
|  |  |  {3,3} = S OK
|  |  |  {4,4} = T OK
|  |  |  {5,5} = A OK
|  |  |  -> palabra completa: RUTA
|  |  + direccion {1,-1}
|  |  |  {1,1} = C OK
|  |  |  {2,0} = "A" != O -> nil
|  |  + direccion {-1,1}
```

Resultados obtenidos:

```
Palabra: ELIXIR
Encontrada: Sí (2 formas)
Forma 1: comienza en {0,0}, termina en {0,5}, dirección derecha
{0,0} -> {0,1} -> {0,2} -> {0,3} -> {0,4} -> {0,5}
Forma 2: comienza en {5,6}, termina en {0,6}, dirección arriba
{5,6} -> {4,6} -> {3,6} -> {2,6} -> {1,6} -> {0,6}

Palabra: COSTA
Encontrada: Sí (1 forma)
Forma 1: comienza en {1,1}, termina en {5,5}, dirección abajo-derecha
{1,1} -> {2,2} -> {3,3} -> {4,4} -> {5,5}

Palabra: PYTHON
Encontrada: No (descartada: la sopa no tiene H, Y)


Palabras encontradas: 2 de 3
```

Reflexión uso IA:

1. En el ejercicio número 3 se utilizó la IA de gemini para la solución de las pruebas de escritorio, mejorar la recursividad que habíamos propuesto nosotros como grupo, y además le preguntamos qué funciones podemos utilizar para la redacción del código que se nos facilitara.

2. ¿Qué funciones se utilizan en Elixir para abrir y cerrar un archivo ?

   Respuesta que nos dio la IA:

   `File.open!/2`: Abre un archivo en el sistema operativo y retorna directamente un descriptor de archivo. El signo de exclamación (`!`) indica que es una función que "lanza excepción": si el archivo no se puede abrir (por permisos o ruta inválida), el programa detiene su ejecución (*crash*) de inmediato en lugar de retornar una tupla de error `{:error, motivo}`.
   `File.close/1`: Cierra el archivo previamente abierto recibiendo su descriptor. Es indispensable ejecutarlo al finalizar las operaciones de E/S para liberar la memoria RAM y los recursos del sistema operativo asignados al flujo de datos.

   Lo que decidimos hacer con esta respuesta fue implementarla en nuestro código, ya que nos servía de mucha ayuda para la generación de los archivos de salida, además nos dimos cuenta de que si no se invoca `File.close/1` al finalizar cada proceso de escritura, los flujos quedan abiertos en la memoria RAM.

3. La sugerencia de la IA consistió en la especificación explícita del átomo de codificación **`:utf8`** dentro de las opciones de apertura de archivos en la función `File.open!/2`, utilizando la tupla de configuración **`[:write, :utf8]`**.

   file_dep = File.open!("depuracion_conexiones.txt", [:write, :utf8])

   Asegura que los reportes generados se puedan visualizar sin alteraciones en cualquier sistema operativo (Windows, macOS, Linux) y en editores como VS Code o Bloc de Notas. Además, garantiza que palabras acentuadas se escriban correctamente, evitando la corrupción de texto.

   Sí, se aplicó en todos los puntos de creación de archivos dentro de `rutas.exs`. Se utilizó para garantizar la presentación impecable de los reportes de salida exigidos en la entrega formal del proyecto, asegurando que los mensajes de error y las trazas de depuración mantuvieran la correcta ortografía.

   Comprender cómo la BEAM administra los canales de E/S (*IO devices*), la diferencia entre escribir bytes crudos y texto UTF-8, y la importancia de la portabilidad en proyectos profesionales.

4. Conclusiones:

   Nosotros entendimos por sí mismos el problema, las dudas que teníamos sobre si el programa debía pedir al usuario el origen y el destino la solucionamos con ayuda del profesor.

   Teníamos algunas dudas sobre la recursividad que al preguntarle a la IA pudimos solucionar y tenerlas de manera muy clara. Además utilizamos IA para entender un poco mejor los árboles de recursión y las pruebas de escritorio. Cuando nos salía un error decidimos acudir a la IA para solucionarlo y él mismo nos daba cosas nuevas que no habíamos visto en clase.

   Lo que aprendimos de este nuevo trabajo son nuevas funcionalidades que podemos aplicarlo para el proyecto final, para nuevos trabajos o ver si en algún nuevo lenguaje se pueden utilizar para hacerlo mucho más eficiente.

   Comprendimos por nosotros mismos el flujo general del programa y cómo plantear los casos base para evitar desbordamientos de memoria. En futuros trabajos, intentaremos planificar mejor los esquemas de tuplas y mapas antes de tirar código, ya que tener claro el formato `{{fila, columna}, letra}` fue lo que hizo posible que la recursividad funcionara.